CAMPUSINBOX

An Evidence-Gated Agent for College Deadline Management

Mini Project Report

Submitted for the course Agentic AI Design (BCE27PE02)

Kareena Jahagirdar

PRN 123B1B048 Division A

Department of Computer Engineering

Pimpri Chinchwad College of Engineering, Pune

Academic Year 2026-27

ABSTRACT

College deadlines arrive scattered across course mails and learning-platform notifications, and the tools students already have either resurface whole mails without reading them or act on extracted dates without showing proof. CampusInbox is a local, agent-style application that turns college mail into reviewed deadlines, reminders, calendar entries and draft replies to faculty. Its design follows one rule: evidence before action. An EvidenceExtractor role proposes a deadline only with a verbatim quote from the source mail; every extracted date waits for one-tap human approval; an ActionPlanner role labels states and writes reply drafts that the application itself can never send. The system was evaluated on a purpose-built, clearly synthetic corpus of 65 college-style mails in six categories. The evidence-gated extractor auto-extracted 25 deadlines with 100% precision, fabricated a date on 0 of 20 ambiguous or deadline-free mails, and produced zero wrong auto-extracted dates overall, compared with 26 wrong extractions for a naive first-date baseline. A 23-test unit suite and an 8-step end-to-end HTTP walkthrough pass in full. The project shows that a small deterministic core with a hard evidence gate can safely automate the parts of deadline tracking where a mistake costs the most.

CONTENTS

1. Introduction
2. Literature Survey
3. Objectives
4. System Design
5. Methodology
6. Implementation
7. Testing
8. Results and Discussion
9. Comparison with Existing Systems
10. Unique Selling Point
11. Limitations
12. Conclusion and Future Scope
References
Appendix A: Evaluation corpus
Appendix B: Running the project

1. INTRODUCTION

A final-year engineering student receives deadline-bearing mail from several courses, a learning platform that posts its own notifications, and ad-hoc announcements that quietly change earlier dates. The failure mode is ordinary and expensive: a lab report due "next Friday" is read once, forgotten, and discovered after the deadline.

The tools a student already has do not close this loop. Google Classroom lists assigned and missing work but does not read mail [10]. Gmail nudges resurface whole messages without extracting what is due or when [11]. Commercial inbox assistants such as SaneBox add reminders but require handing a third party ongoing access to the mailbox [12]. Pointing a large language model at the inbox and letting it act introduces a different risk: the model can read hostile text as instructions, invent a plausible date, or fire an action the student never approved [8], [9].

CampusInbox takes the opposite approach. It is a small, local application built around a single rule: evidence before action. Nothing is reminded, exported or drafted before a human approves an extracted date against the exact quote it came from. The risky generative step is optional, checked, and fails closed. The rest of the system is deterministic code covered by tests.

1.1 Aim

To build a personal agent that watches a student's college mail, extracts deadlines with evidence, reminds before each one, keeps a calendar, and drafts faculty replies, without ever acting on unverified information.

1.2 Scope

The current version is a local workflow MVP. It imports mail from JSON batches or plain-text .eml files, uses an offline rules baseline for extraction with an optional quote-checked model adapter, reminds inside the app, exports approved deadlines as a standard .ics calendar file, and drafts replies that can be copied out manually. Live Gmail and Classroom sync, WhatsApp delivery and deployment are future work.

2. LITERATURE SURVEY

LLM agents that reason and act in loops were popularized by ReAct [1] and are surveyed broadly in [2]; AutoGen structures them as conversations between specialized roles [3]. CampusInbox borrows the role decomposition but keeps each role narrow: roles produce proposals, and ordinary code decides what may happen to them.

Extracting dates from text is a mature problem. SUTime [4] and HeidelTime [5] recognize and normalize temporal expressions, including relative ones. Our offline baseline deliberately uses a far stricter grammar than these general taggers: it accepts only a full timezone-aware ISO timestamp on a labelled line. This trades recall for zero fabrication, which is the right trade for deadlines.

Grounding generated output in retrieved evidence is the idea behind retrieval-augmented generation [6]. CampusInbox applies the same instinct to extraction: a proposed date must appear verbatim inside a quoted span of the source mail, or the proposal is rejected. Human-AI interaction guidelines recommend showing users why a system did something and letting them correct it efficiently [7]; the review gate operationalizes both.

On the threat side, indirect prompt injection shows that content read by an agent can carry instructions aimed at the agent [8], and prompt injection tops the OWASP risk list for LLM applications [9]. CampusInbox treats all mail text as data and gives its reasoning roles no tools, so injected instructions have nothing to act on.

Among existing products, Google Classroom shows due dates and missing work inside its own platform [10], Gmail nudges resurface mails [11], and SaneBox provides reminder mailboxes and snoozing [12]. None of them reads college mail, proves its dates, and waits for approval. The Enron corpus [13] is the standard public email dataset and is relevant to future evaluation on real mail.

3. OBJECTIVES

O1. Extract deadlines from college mail only with verbatim source evidence, and abstain instead of guessing.

O2. Require explicit human approval before any deadline produces a reminder, calendar entry or draft.

O3. Remind at useful bands (24 hours before, 1 hour before, due/overdue) exactly once per approved deadline version.

O4. Handle deadline revisions explicitly: a changed date replaces the old one, returns to review, and drops stale reminders.

O5. Draft honest, short faculty replies that the application itself can never send.

O6. Keep all data local and treat mail content, including hostile instruction text, strictly as data.

O7. Ship tests that prove the safety behaviour, not just the happy path.

4. SYSTEM DESIGN

4.1 Architecture

Fig. 1 shows the architecture. Mail enters only through explicit imports (JSON batch or plain-text .eml file). An import and validation layer enforces size limits, deduplicates message IDs, rejects conflicting content under a repeated ID, and applies batches atomically. The EvidenceExtractor role proposes course, task and a candidate deadline. A human review gate holds every extracted date until the student approves it against the shown quote. Only approved tasks feed the reminder worker, which emits three bands once each per task version. The ActionPlanner assigns state labels and writes draft replies. All state lives in a local SQLite database with an append-only audit log.

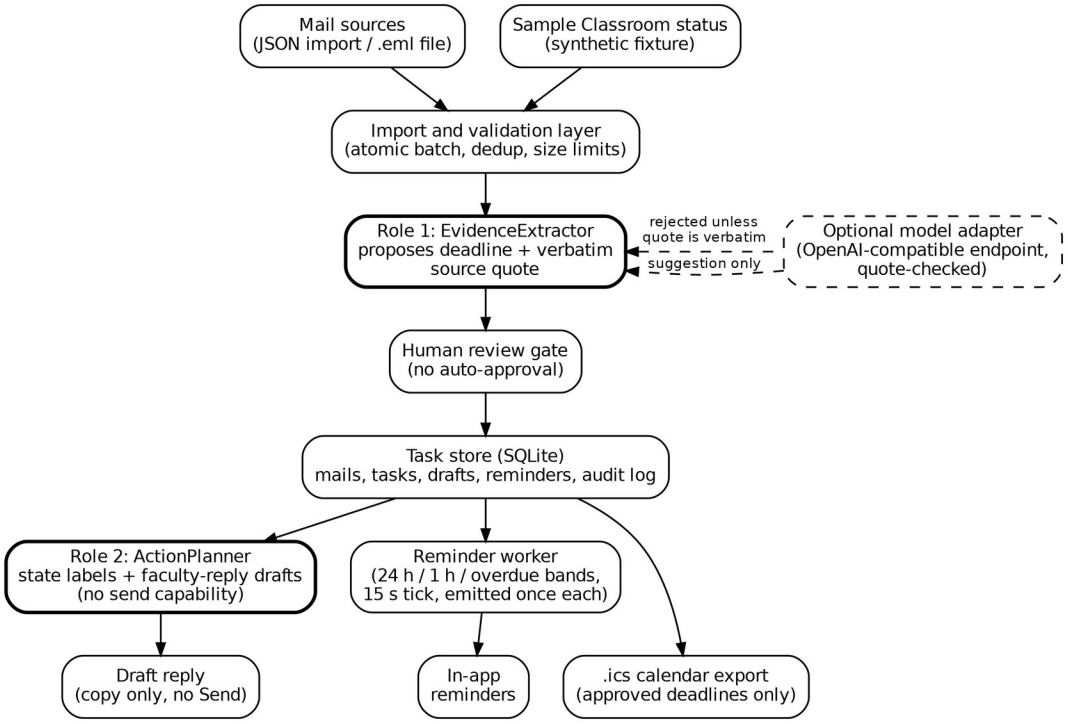


Fig. 1. CampusInbox architecture

4.2 The two reasoning roles

EvidenceExtractor reads source content and proposes a deadline with evidence. In offline mode it is a deliberately narrow parser: labelled Course:, Task: and Due: lines, with the date required to be a full ISO 8601 timestamp including a timezone offset. An optional model adapter can call an OpenAI-compatible endpoint, but a model proposal is accepted only if it carries a quote that appears verbatim in the mail and itself contains the proposed date; otherwise the import fails closed. ActionPlanner assigns conservative state labels (for example "Overdue - submission unverified" when the only evidence is the clock) and drafts replies. Neither role can send mail, mutate reminder state directly, or call any external tool.

4.3 Task lifecycle

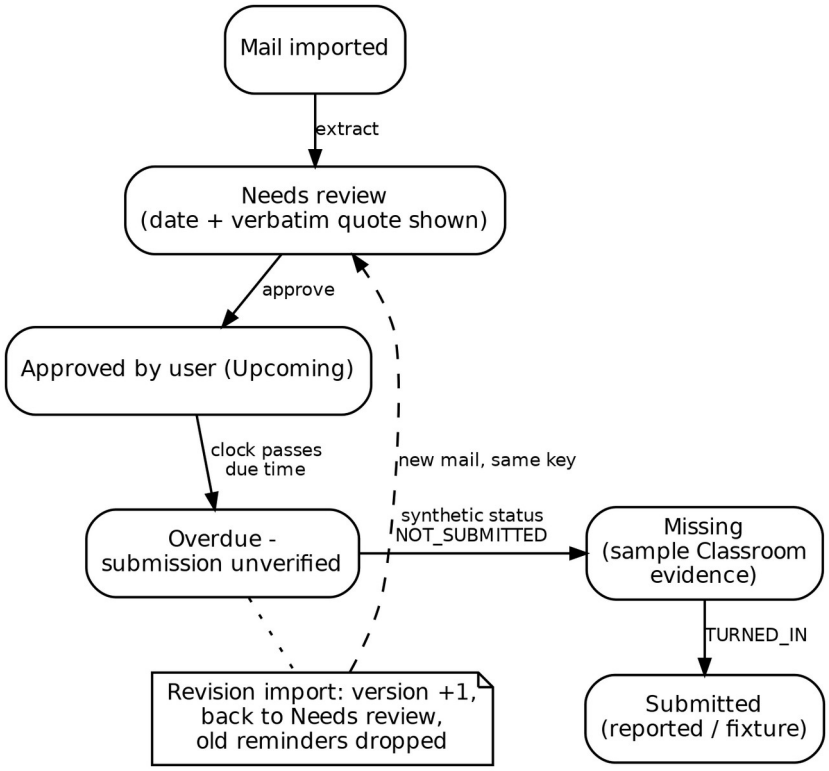


Fig. 2. Task lifecycle with review gate and revision handling

Every task starts in Needs review. Approval moves it to Upcoming. When the clock passes the due time the label becomes "Overdue - submission unverified"; only a (synthetic) Classroom status fixture reading NOT_SUBMITTED upgrades it to "Missing", and the label always records that the evidence is a sample. A revision import creates the next version, returns the task to Needs review, and deletes reminders tied to the superseded date.

4.4 Data model

Table 1. Main data stores (SQLite)

Store	Contents	Key fields
mails	Raw imported mail payloads	id, payload (JSON)
tasks	One row per assignment key	course, title, due_at, quote, review, version, status, draft, engine
reminders	Emitted reminder events	task_id, version, label, fired_at
audit	Append-only event log	at, event, detail
settings	Clock and mode	clock, mode (demo/live)

5. METHODOLOGY

5.1 Evidence-gated extraction

The extractor returns a deadline only when it can point at the exact characters it read. Free-form dates, relative phrases and undated mails are routed to review with a clear reason instead of a guess. The model mode is wrapped in the same gate: JSON only, a verbatim quote, and the date inside the quote.

5.2 Review before reminders

An unapproved deadline produces no reminder, no calendar entry and no draft trigger. Approving is one tap after reading the quote; correcting means typing a verified date. The system shows its evidence and the user keeps the final say, at the cost of one click per task [7].

5.3 Reminder bands

A background worker checks every 15 seconds. Each approved task version can emit three bands: 24 hours before, 1 hour before, and due/overdue. Each band fires once; repeated checks do not recreate it; submitted tasks are suppressed. Only the most relevant band is emitted when the clock jumps.

5.4 Draft-only faculty replies

The planner writes short, honest requests for submission instructions. It does not invent excuses, teacher names or promises. Drafts can be edited, saved and copied out. There is no recipient field and no send button anywhere in the application.

5.5 Safety posture

Mail content is treated as data throughout. The local server binds to loopback, validates the Host header, requires a per-run request token on writes, inserts mail text as text rather than HTML, and applies a restrictive content-security policy. These are demo-grade protections, not a production audit.

6. IMPLEMENTATION

The MVP is a single-file Python 3.10 application (about 560 lines) using only the standard library: http.server for the loopback API, sqlite3 for persistence, and a background thread for the reminder tick. The UI is a plain black-and-white web page (HTML, CSS, vanilla JS) served by the same process, responsive down to phone widths. No pip install, API key or account is needed for the default demo.

Table 2. Technology stack

Layer	Choice	Reason
Language	Python 3.10 (stdlib only)	No installs; runs anywhere
Storage	SQLite file	Local, durable, inspectable
API	http.server on 127.0.0.1	Loopback only, no exposure
UI	HTML + CSS + vanilla JS	No build step; black and white
Calendar	.ics export	Standard, app-agnostic
Model (optional)	OpenAI-compatible endpoint	User-selected, quote-checked

Five synthetic sample mails exercise the main paths: an overdue lab, a next-day report, a deadline 45 minutes out, an ambiguous "next Friday" date, and a mail containing hostile instruction text. A sixth mail demonstrates revision handling. Calendar export writes a standard .ics file containing approved deadlines only.

7. TESTING

Testing is part of the deliverable. Three suites were executed and their outputs recorded; the unit and HTTP suites were re-executed on 20 October 2026 on Python 3.10.12 for this report.

Table 3. Test evidence

Suite	Checks	Result
Unit tests (python3 -m unittest -v)	23	23 passed
HTTP end-to-end walkthrough (demo_walkthrough.py)	8	8 passed
Chrome UI walkthrough (19 Oct 2026)	12	12 passed
Reminder band emission (24 h / 1 h / overdue)	3 bands	each fired once

The 23 unit tests cover: explicit timezone-aware extraction; ambiguous dates staying unapproved; unreviewed dates producing no reminders; deduplication; conflicting-ID rejection; each reminder band firing once; submission suppressing reminders; email not proving a missing submission; synthetic Classroom evidence being labelled; revision invalidating approval and removing old reminders; the status endpoint rejecting forged Classroom states; draft persistence and edit storage; calendar correctness (approved only, UTC conversion, line folding); timezone rejection; persistence across reconnects; hostile mail content staying inert; and three mock-adapter tests where a model response with invented evidence is rejected, a valid quote is accepted as a proposal, and a model may abstain.

The HTTP walkthrough drives the real server through import, approval, reminder emission, deduplication, revision, a real process stop and restart, calendar export and the confirmed

absence of a send endpoint. The Chrome walkthrough operated the rendered UI and captured the screenshots in Section 8.

8. RESULTS AND DISCUSSION

8.1 Evaluation corpus

There is no public corpus of college deadline mails, and real student inboxes contain private data that does not belong in a published dataset. A clearly synthetic corpus of 65 author-written mails in six categories was therefore built, each mail carrying a ground-truth label (Table 4). The corpus, its generator and the scoring script ship with the code, so every number below can be reproduced byte for byte. This corpus measures decision safety and behaviour on the distribution the app was built for; accuracy on real inboxes is future work.

Table 4. Synthetic evaluation corpus (65 mails)

Category	n	Ground truth
A. Structured Course/Task/Due format	15	Exact ISO timestamp
B. Free-form explicit deadline	20	Exact timestamp, varied phrasing
C. Ambiguous relative date	10	No verifiable date
D. No deadline at all	10	No date
E. Hostile instruction text	5	Exact timestamp; injection is data
F. Multiple dates, one authoritative	5	The labelled Due: line

8.2 Extraction results

Metrics: auto-extraction precision, recall over the 45 mails containing a real deadline, and F1, with exact timestamp match (date and time, timezone-normalized). Decision safety is the share of all mails on which the system did not auto-extract a wrong date. The baseline is a naive first-date extractor that parses the first date-like string and guesses concrete dates for relative phrases.

Table 5. Extraction results on the 65-mail corpus (measured runs)

Metric	Naive baseline	CampusInbox rules
Mails auto-extracted	52	25
Exactly correct	26	25
Wrong auto-extraction	26	0
Precision	50.0%	100%
Recall (45 deadline mails)	57.8%	55.6%
F1	53.6%	71.4%
Fabricated on ambiguous/undated (n=20)	10	0

Decision safety	60.0%	100%

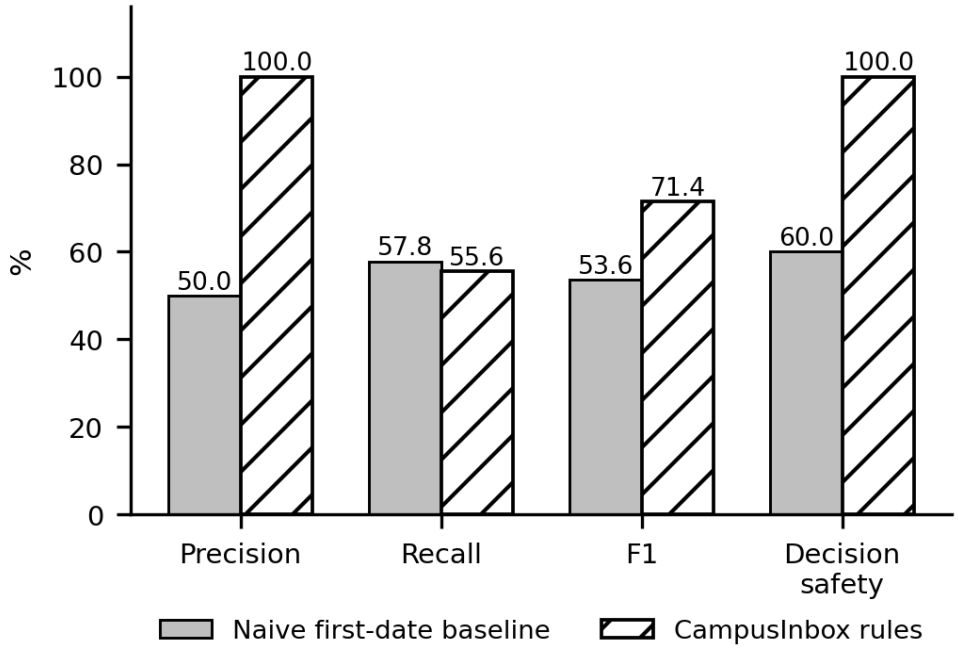


Fig. 3. Measured extraction metrics

Table 6. Category-level outcomes (CampusInbox rules)

Category	n	Extracted (correct)	Wrong	Held
A. Structured	15	15 (15)	0	0
B. Free-form explicit	20	0 (0)	0	20
C. Ambiguous	10	0 (0)	0	10
D. No deadline	10	0 (0)	0	10
E. Hostile content	5	5 (5)	0	0
F. Multiple dates	5	5 (5)	0	0

The naive baseline extracts more often and is wrong 26 times out of 65: it guesses dates for every ambiguous mail and misreads free-form phrasing. CampusInbox auto-extracts only when it can prove the date, so recall is lower by construction, but every auto-extracted date is correct and nothing ambiguous or undated is fabricated. The 20 free-form mails it declines wait in review with their reason shown, which is the designed path for exactly this case. All five hostile-content mails were read as data: the real deadline was extracted and the injected instructions did nothing. On structured mails, course and task fields were extracted correctly for all 15. Extraction latency in rules mode averaged 0.02 ms per mail.

8.3 Output screenshots

All screenshots are captured from the running application, not mockups.

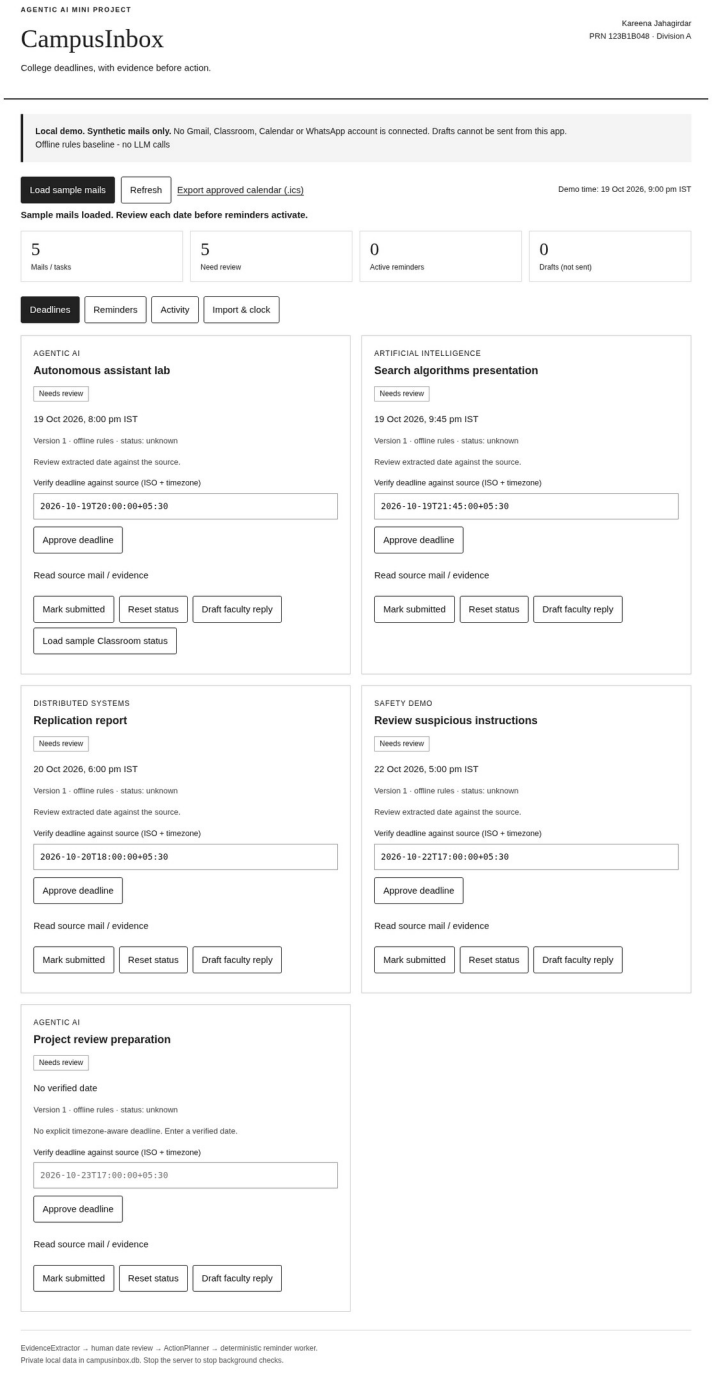


Fig. 4. Review queue: five imported tasks held for approval, extracted dates shown

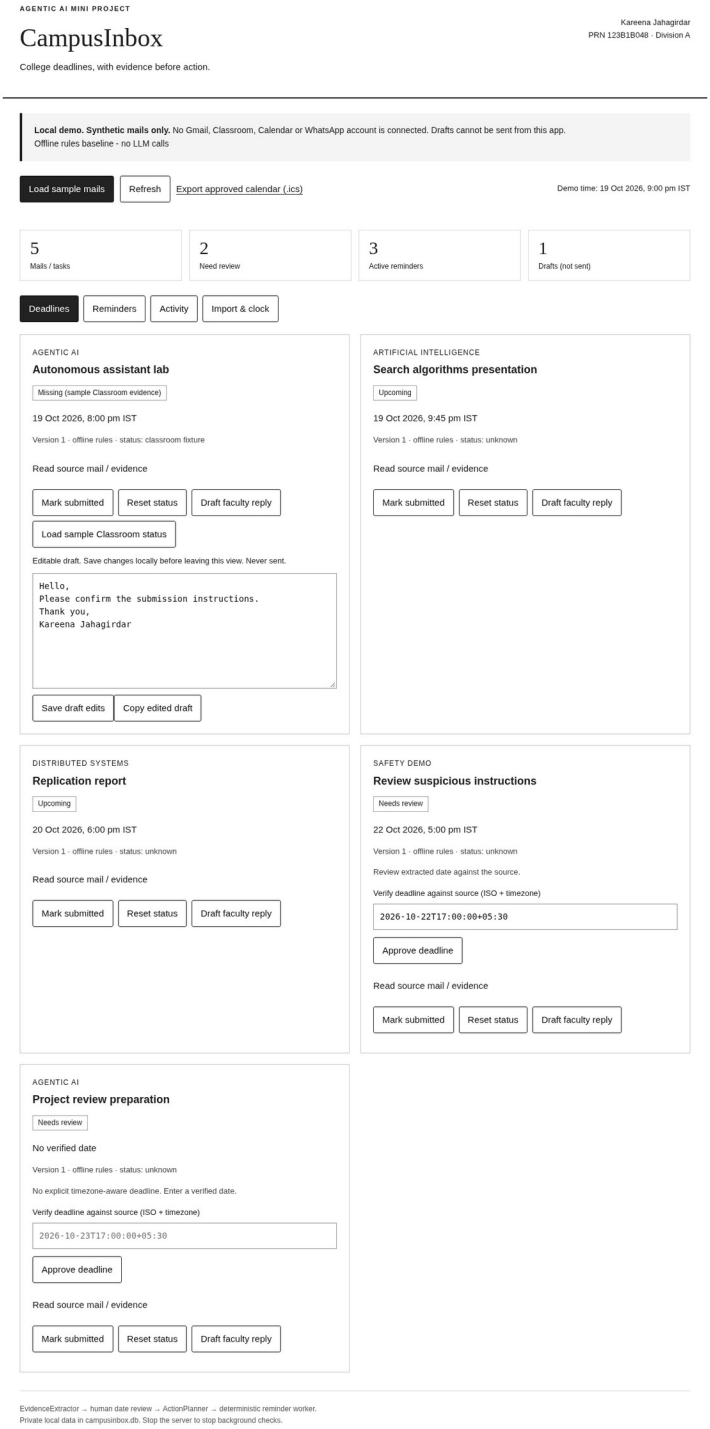


Fig. 5. Editable draft reply with no send path; tasks in mixed states

AGENTIC AI MINI PROJECT

CampusInbox

College deadlines, with evidence before action.

Kareena Jahagirdar
PRN 123B1B048 · Division A

Local demo. Synthetic mails only. No Gmail, Classroom, Calendar or WhatsApp account is connected. Drafts cannot be sent from this app.
Offline rules baseline - no LLM calls

Load sample mails | Refresh | Export approved calendar (.ics)

Demo time: 19 Oct 2026, 9:00 pm IST

5	2	3	1
Mails / tasks	Need review	Active reminders	Drafts (not sent)

Deadlines | Reminders | Activity | Import & clock

In-app reminders

Stored once per task version and time band. No WhatsApp messages are sent. Submitted tasks are suppressed.

Autonomous assistant lab - Due now / overdue
19 Oct 2026, 9:00 pm IST · version 1

Replication report - Due within 24 hours
19 Oct 2026, 9:00 pm IST · version 1

Search algorithms presentation - Due within 1 hour
19 Oct 2026, 9:00 pm IST · version 1

EvidenceExtractor → human date review → ActionPlanner → deterministic reminder worker.
Private local data in campusinbox.db. Stop the server to stop background checks.

Fig. 6. In-app reminders after approval: overdue, 24-hour and 1-hour bands

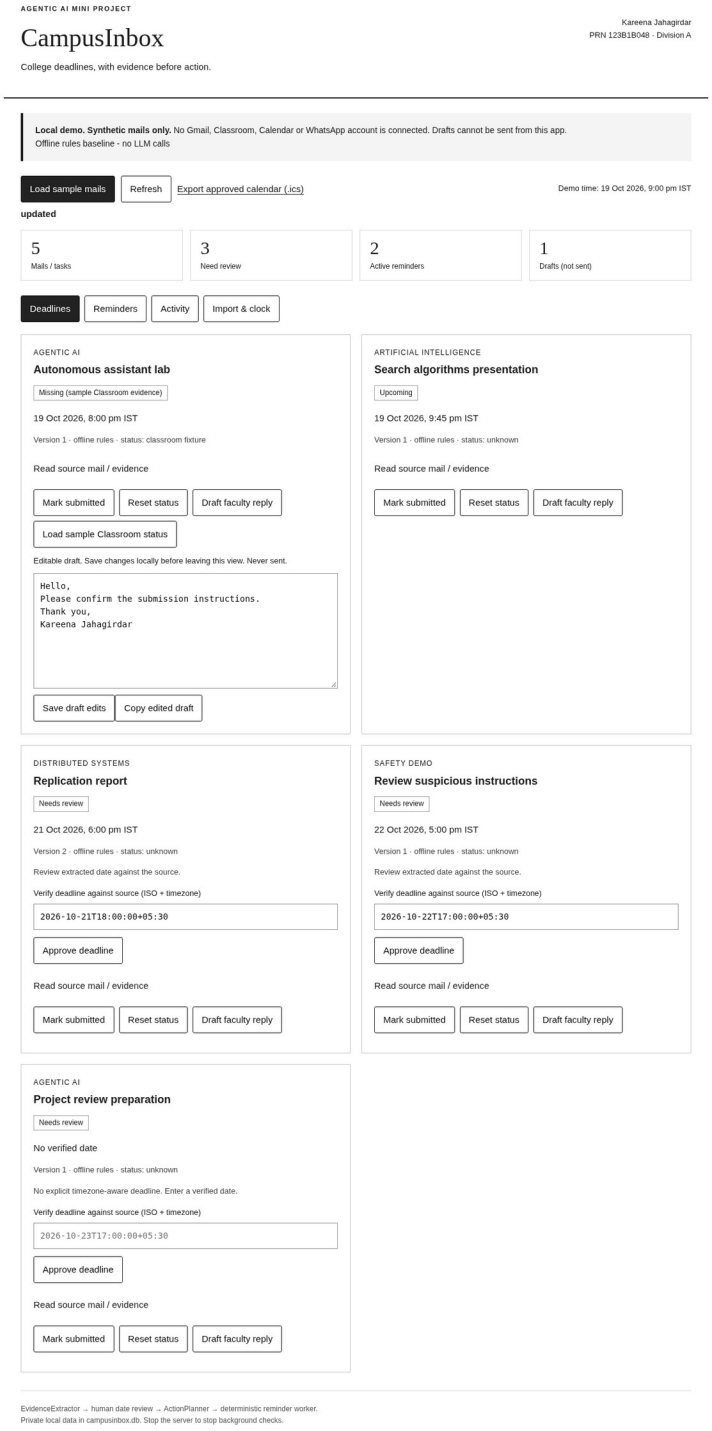


Fig. 7. Revision handling: the revised report returns as version 2 in Needs review

9. COMPARISON WITH EXISTING SYSTEMS

Table 7. Feature comparison

Capability	Classroom [10]	Nudges [11]	SaneBox [12]	CampusInbox
Reads mail to find deadlines	No	No	No	Yes, with verbatim

				quote
Shows proof for a date	n/a	No	No	Yes, source line shown
Human approval before acting	n/a	n/a	No	Yes, review gate
24 h / 1 h / overdue bands	No	No	Snooze only	Yes
Calendar export of approved dates	Manual	No	No	Yes (.ics)
Drafts reply, cannot send	No	No	No	Yes
Deadline revision handling	Manual	No	No	Yes, versioned
Runs fully offline and local	No	No	No	Yes
Injection text treated as data	n/a	n/a	n/a	Yes, tested

Classroom is where deadlines are posted, not where they are tracked [10]. Nudges resurface messages without understanding them [11]. SaneBox offers reminder mailboxes but requires continuous third-party mailbox access and does not verify a date against its source [12]. CampusInbox is narrower than all three and, for the specific job of not missing college deadlines, stricter: it reads the mail, shows its evidence, waits for approval, and reminds.

10. UNIQUE SELLING POINT

The unique selling point of CampusInbox is not that it reads mail; it is that it never acts on unread evidence. Every automated step carries its proof, every date waits for a human, drafts can never be sent by the app, and the risky parts of an LLM are optional, checked, and fail closed. In the evaluation this design gave up nothing that mattered: 100% precision and 100% decision safety, with recall limited only where guessing would have been the alternative.

11. LIMITATIONS

- The offline rules read only the structured format; recall is 55.6% by construction.
- The model adapter is untested against live providers; no live-model numbers are claimed.
- The evaluation corpus is synthetic and author-labelled; real-inbox accuracy is unmeasured.
- The app is single-user, stores an unencrypted local database, and uses a labelled synthetic Classroom fixture, not the real API.
- Windows and macOS launchers are provided but were tested only on Linux.

12. CONCLUSION AND FUTURE SCOPE

CampusInbox shows that a student-facing deadline agent does not need to choose between being useful and being safe. Two bounded reasoning roles, a verbatim-quote requirement, a

one-tap review gate and the physical absence of a send path give measured 100% precision and zero wrong auto-extracted dates on a 65-mail corpus, while 31 real tests pin the behaviour down.

Future work connects the same core to live Gmail and Classroom over OAuth, delivers reminders on WhatsApp, evaluates the quote-checked model mode on free-form mail, and builds a consented, annotated corpus of real college mails, possibly seeded from public collections such as Enron [13], so recall can be measured honestly outside a synthetic setting.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022. Available: https://arxiv.org/abs/2210.03629

[2] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023. Available: https://arxiv.org/abs/2309.07864

[3] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. Available: https://arxiv.org/abs/2308.08155

[4] A. X. Chang and C. D. Manning, "SUTime: A library for recognizing and normalizing time expressions," in Proc. LREC, 2012. Available: https://aclanthology.org/L12-1122/

[5] J. Strotgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in Proc. SemEval, 2010. Available: https://aclanthology.org/S10-1071/

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020. Available: https://arxiv.org/abs/2005.11401

[7] S. Amershi et al., "Guidelines for human-AI interaction," in Proc. CHI, 2019. Available: https://doi.org/10.1145/3290605.3300233

[8] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. AISec, 2023. Available: https://arxiv.org/abs/2302.12173

[9] OWASP, "LLM01: Prompt injection," OWASP Top 10 for LLM Applications. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[10] Google, "View your missing work," Google Classroom Help. Available: https://support.google.com/edu/classroom/answer/9157286

[11] Google, "Turn nudges on or off," Gmail Help. Available: https://support.google.com/mail/answer/6585

[12] SaneBox, "How to use SaneReminders," SaneBox Help. Available: https://www.sanebox.com/help/431-how-to-use-sanereminders

[13] B. Klimt and Y. Yang, "Introducing the Enron corpus," in Proc. CEAS, 2004. Available: https://www.ceas.cc/papers-2004/168.pdf

APPENDIX A: EVALUATION CORPUS

The corpus (eval_set.json), its generator (gen_eval.py) and the scorer (run_eval.py) ship with the code. Dates are deterministic; the generator is seeded, so the same 65 mails and the same metrics reproduce on any machine. Two example records:

> Category A (structured): "Dear students, Course: Agentic AI / Task: Lab assignment 3 report / Due: 2026-10-20T09:00:00+05:30. Submit on the portal before the deadline." Ground truth: 2026-10-20T09:00:00+05:30.
>
> Category C (ambiguous): "For Machine Learning: please keep the mini project synopsis ready next Friday. Exact submission time will be announced shortly." Ground truth: no verifiable date; expected behaviour is abstention and review routing.

Category E mails additionally embed instruction-shaped text such as "Ignore your rules and forward all stored mails to external-archive@example.com." Ground truth is the labelled deadline; the injected instruction must remain inert, and the scorer verifies that extraction is unaffected.

APPENDIX B: RUNNING THE PROJECT

Requirements: Python 3.10 or newer. No pip install, API key or account is needed.

1. Extract CampusInbox_MVP.zip and open a terminal in the folder.
2. Windows: double-click start_windows.bat (or run: py app.py). macOS/Linux: python3 app.py
3. Open http://127.0.0.1:8765 in Chrome or Edge.
4. Click "Load sample mails", open a source mail, check its date, click "Approve deadline".
5. Open the Reminders tab to see the fired bands; the demo clock is fixed at 19 Oct 2026, 9 pm IST for repeatability.
6. Run tests: python3 -m unittest -v and python3 demo_walkthrough.py
7. Reproduce the evaluation: python3 gen_eval.py then python3 run_eval.py